TECHNICAL ARCHITECTURE / TRACK 3

Iris: one Brain, multiple Bodies.

A capability protocol that lets a persistent Hermes agent discover, route to, and recover across the devices a user already owns.

IMPLEMENTATION

The core idea

The cloud Brain holds memory and orchestration. Devices join as Bodies and contribute live capabilities: local inference, voice, notifications, location, and native interaction.

This deck explains the system below the interface: protocol, transport, nodes, routing, compute, and tenant isolation.

The Brain is persistent. Bodies are opportunistic.

The data plane keeps the agent reachable 24/7 while the capability plane expands or contracts as devices connect.

LOCAL MACHINE

Desktop Body

Electron desktop detects local services and announces GPU LLM and F5-TTS voice endpoints.

MOBILE

Android Body

Kotlin foreground service maintains a Ktor WebSocket. It announces notification and location.

WRIST

Wear OS client

Native watch client connects to the Hermes gateway for voice, TTS, sessions, and paired-phone credential sync.

CLOUD BRAIN

Hermes Brain

Gateway WebSocket, live registry, orchestrator, memory, tools, and model route selection.

Each Body owns its hardware and permissions. The Brain owns intent, context, tool use, and execution policy.

A small protocol turns devices into an agent capability fabric.

Bodies announce what is available after connection, then receive actions over the same authenticated gateway WebSocket.

WIRE FORMAT

```
hermes.capabilities.announce
{
  device: 'android',
  notification: { available: true },
  location: { available: true }
}
```

WIRE FORMAT

```
notification.send.request
{ title, body, request_id }

device.action.response
{ request_id, success, device }
```

Two patterns, one transport

Push actions: notification.send. Pull actions: location.current returns latitude, longitude, and accuracy in the correlated response envelope.

Presence is a routing signal, not a settings screen.

The Capability Registry is updated by announce and disconnect events. The Orchestrator receives a live provider chain for every capability.

ON CONNECT

1. Register

An announce payload is namespaced by capability and bound to a device type. Registry state changes without redeploying the Brain.

PER ACTION

2. Select

For each action, the Orchestrator chooses the best provider in its configured, registry-aware chain.

ON WIRE

3. Correlate

A request_id follows the action from Brain to Body and back. The first valid response completes the call.

FAILOVER

4. Recover

Disconnect removes the provider. Failure or timeout advances to the next route for a future turn.

```
registry + config -> ordered providers -> action -> correlated result
```

The Node never starts with an open socket.

Authentication happens before the persistent connection. The gateway hands the Body a single-use WebSocket ticket after login.

01

Login

POST /auth/password-login with the selected provider.

02

Ticket

POST /api/auth/ws-ticket returns a short-lived, single-use ticket.

03

Socket

wss://gateway/api/ws?ticket= establishes the authenticated session.

04

Announce

The Body reports its current capabilities after every reconnect.

The ticketed socket keeps the action channel authenticated without placing a long-lived credential in every message.

Android exposes real-world context only when the Brain needs it.

The app is Kotlin-based, uses Ktor for the gateway WebSocket, and maintains the Body with a foreground service.

KOTLIN + KTOR

Lifecycle

Login, fetch a WebSocket ticket, connect, announce, and repeat announce on reconnect. It is designed for a persistent mobile session.

PUSH

notification.send

The Brain emits a request. Android posts a high-priority OS notification then answers device.action.response.

PULL

location.current

Android uses LocationManager.getCurrentLocation and returns a live GPS fix only after the Brain requests it.

HARDWARE RUN

Validated

Galaxy S20 FE, production gateway, real notification delivery and live GPS response.

Important: location is not streamed to the cloud. It is requested on demand through the capability action.

The desktop contributes private compute and a voice persona.

The Electron desktop watches local services, checks their health, and announces available endpoints to the Brain.

WIRE FORMAT

```
device: 'desktop'
speech: {
  available: true,
  localUrl: '127.0.0.1:8123',
  voices: ['dot']
}
llm: { localUrl: '127.0.0.1:1234/v1' }
```

INFERENCE

Local LLM

Gemma runs in LM Studio on the Radeon RX 9060 XT. The local endpoint is OpenAI-compatible.

SYNTHESIS

Voice

F5-TTS runs beside the model and exposes Dot, the secretary persona, as the speech provider.

The Brain can use the machine when it is online. When it is not, the routing chain advances instead of losing the assistant.

A native wrist client for the same Brain.

The current Wear OS implementation prioritizes direct interaction: short voice exchanges, spoken responses, sessions, and seamless sign-in from the paired phone.

TRANSPORT

Gateway client

Ktor client mirrors the phone connection model: auth, ticket, and a persistent WebSocket to Hermes.

INTERACTION

Voice UX

Native speech-to-text and TTS make the watch a low-friction conversational surface rather than a notification mirror.

PAIRING

Data Layer

Paired-phone credential synchronization reduces setup friction while preserving a direct watch-to-Brain session.

HONEST STATUS

Scope today

The watch is a direct client. Sensor and heart-rate capabilities are a planned capability-protocol extension.

One inference contract across local, cloud, and fallback routes.

Every route is OpenAI-compatible, so Hermes can change provider without changing the agent interface or conversation state.

DESKTOP TIER

Local AMD

Radeon RX 9060 XT 16 GB runs Gemma through LM Studio. First choice when the desktop is live and privacy matters.

CURRENT RUNTIME

AMD cloud

The live pod uses an MI300 and vLLM. Gemma is served at /v1 with GPU memory utilization and long-context configuration.

WORKSTATION

AMD validation

Radeon PRO W7900 was tested during the hackathon with Gemma Q4 served through llama.cpp.

LAST RESORT

Fallback

Fireworks AI API is wired as the final external resilience route.

RX 9060 XT / LM Studio -> MI300 / vLLM -> Fireworks API

The Brain keeps the conversation; compute can move.

Routing is about available capability and execution policy, not moving the user's agent identity from one model host to another.

BRAIN

1. Intent

Hermes receives the turn with its memory, tools, and conversation context.

POLICY

2. Resolve

The Orchestrator observes live providers and selects a model route.

COMPUTE

3. Execute

The chosen OpenAI-compatible endpoint generates the response or tool call.

CONTINUITY

4. Recover

If the route disappears, the next turn uses the next available provider.

This separation is what lets Iris preserve an always-on personal assistant while still harvesting the user's own GPU when it becomes available.

The product path is implemented, not mocked.

Iris combines sign-up, Stripe checkout, automated provisioning, and isolated Hermes environments so the architecture can be delivered as a service.

01

Account

Django account and console establish the customer identity.

02

Checkout

Stripe supports managed-compute and BYOK plan paths.

03

Provision

Ubuntu automation creates the tenant environment.

04

Operate

The new Brain receives its own services and can attach Bodies.

Isolation boundary per tenant

Unique Unix identity, Hermes runtime, memory, credentials, environment variables, and services. Tenant setup also uses a per-tenant dashboard secret for server-to-server trust.

Capabilities become tools without modifying the agent core.

The iris-body skill invokes the Orchestrator through loopback HTTP. The transport and device protocol remain behind a narrow integration surface.

WIRE FORMAT

```
POST /iris/notify
  -> notification.send.request
  -> device.action.response
  -> 200 { delivered: true }

GET /iris/location
  -> location.current.request
  -> 200 { latitude, longitude, accuracy }
```

DECOUPLING

Agent contract

The agent calls a simple local HTTP endpoint. It does not need to know whether the Body is a phone today or another device tomorrow.

OPERATIONAL

Failure contract

If no Body announced the capability, endpoints return HTTP 503 with a clear reason instead of pretending success.

The same pattern can extend to future Bodies and new capability types without creating a custom agent integration for every device.

What is running, what is deliberately next.

The implementation is centered on a working end-to-end vertical slice, then expands the capability vocabulary with the same protocol.

IMPLEMENTED

Working now

Production Brain, authenticated WebSocket, registry and orchestration flow, Android notification/location, desktop local services, MI300 vLLM runtime, checkout, and tenant provisioning.

EVIDENCE

Validated

Android ran against the production gateway on a Galaxy S20 FE. Notification delivery and a live GPS fix were returned end-to-end through the agent workflow.

ROADMAP

Next protocol extensions

Stable per-Body identities, versioned capability schemas, granular permissions, streaming results, and Watch sensor capabilities.

Iris is the door through which Hermes can use the real world - while the user retains control of the devices behind it.